Guideline Session - Testing

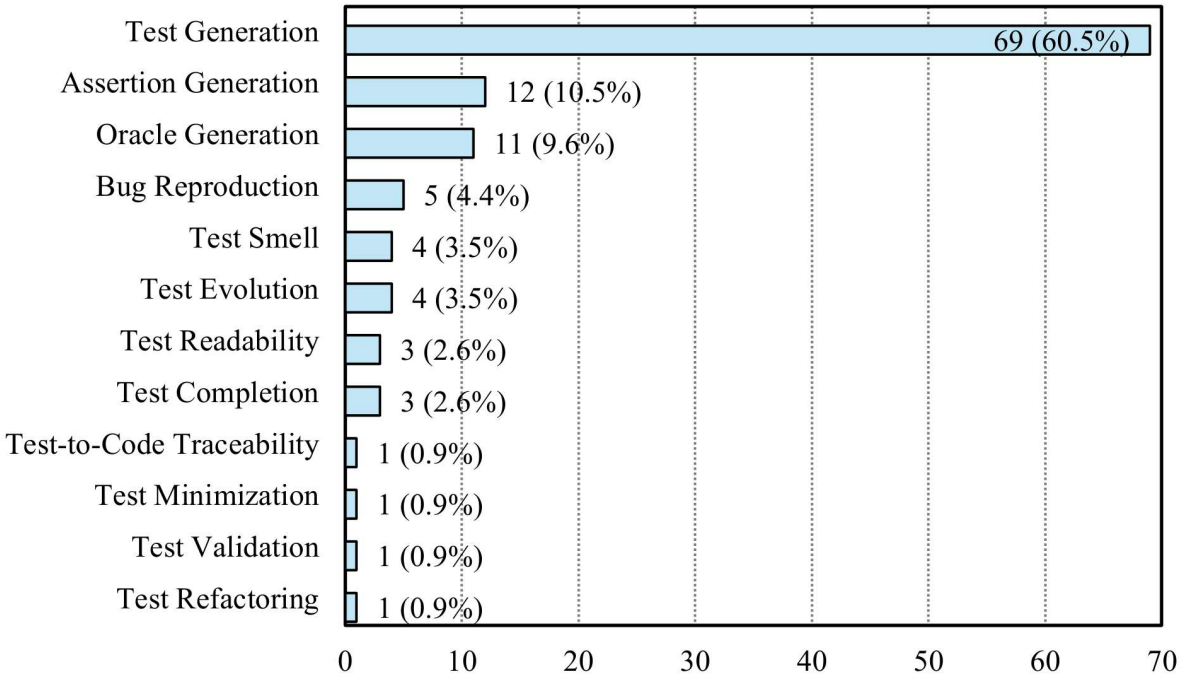
CS 630 - Generative Software Engineering

The Testing Team: Julian Maurus, Sebastian Menzel, Simon Wally

22.04.2026

Introduction

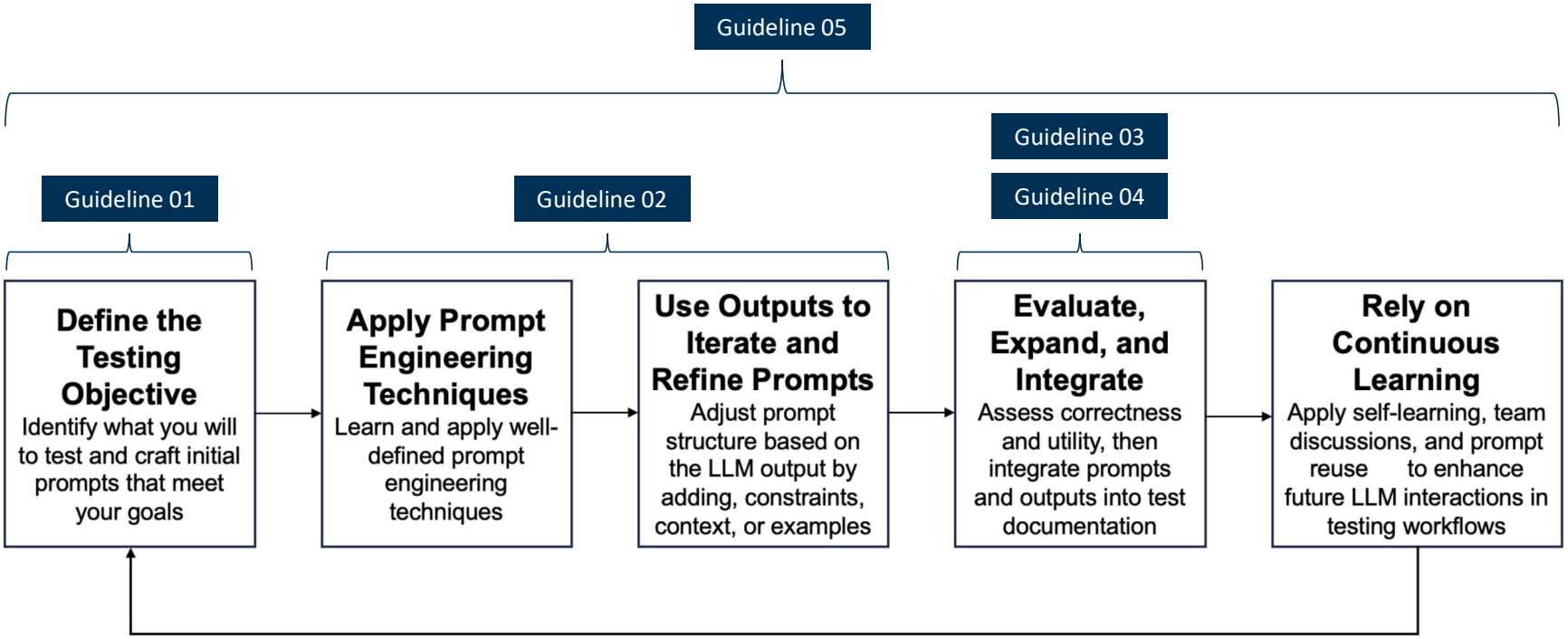
Where People use LLMs in Software Testing?



(Zhang et al., 2025)

Overview

LLM into Software Testing



(Santana et al., 2025)

Guideline 01

Define the Testing Objective

Identify the Testing Objective and Expected Artifact
Clearly specify what testing goal you want to achieve and which artifact you expect as output. Name the exact target function/module and when known the intended scenario set.

Establish Scope Boundaries
Define the scope explicitly by identifying which functions/features are in scope and which are out of scope. Articulate negative paths, edge cases and boundary conditions you want to cover. For security-critical targets, explicitly include required security scenarios.

Anchor Success Criteria
Ground success criteria in concrete, verifiable sources: acceptance criteria from user stories, requirement documents, real-world examples and past test data. Define measurable quality constraints and required technical practices to reduce redundancy and improve maintainability.

Guideline 01

Define the Testing Objective

```
Test Objective: Generate unit tests for the
apply_discount(price, is_vip) function

Expected Artifact: PyTest test file with parametrized tests

Scope:
  - In: Boundary testing ($100), VIP logic, discount calculation
  - Out: Integration with payment systems

Success Criteria:
  - Acceptance: Function returns 20% discount only
  when price > 100 AND is_vip == True

  - Mutation: Test suite achieves >80% mutation score
  (kills high-order mutants)

Edge Cases:
  - Boundary: price == 100 (exactly at threshold)
  - Negative: non-VIP users, prices <= $100
```

Note: This example represents a structured test specification. It is the conceptual input used to inform prompt engineering, not the final prompt text itself.

When to Apply?

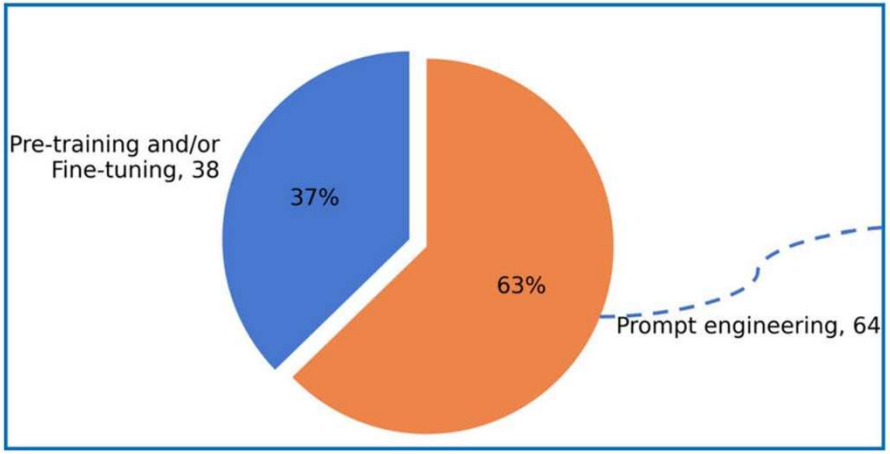
- During the design phase**, before any prompt execution** or **automated test code is generated**
- When defining the baseline for test suite evaluation
- When **multiple stakeholders** are involved and clarity is needed
- When mutation testing or coverage metrics will be used
- During **requirement analysis** or **test planning phases**

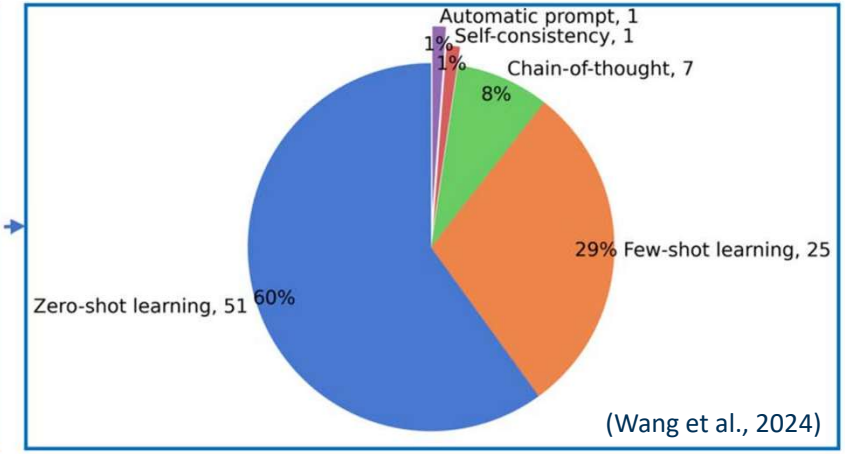
When to Avoid?

- **Trivial utility functions** where informal scoping is sufficient
- **Exploratory testing** phases where scope naturally evolves
- Legacy codebases with **undefined or conflicting requirements**
- Scenarios with **missing functional context** or **incomplete code**

Guideline 02

Apply Prompt Engineering Techniques





(Wang et al., 2024)

"Prompt engineering is the practice of crafting precise prompts to help generative artificial intelligence (AI) models correctly respond to questions and perform a wide range of tasks. This practice improves the model's ability to produce accurate and relevant responses."

(SAP, 2024)

- **Zero-Shot:** Providing a task with **no examples**. Best for simple and generic tasks.
- **Few-Shot:** Providing **2–5 examples** of the desired input/output format. Best for consistency and domain-specific behavior
- **Chain-of-Thought (CoT):** Asking the model to **"think step-by-step."** Best for for reasoning-heavy tasks.

Guideline 02

Apply Prompt Engineering Techniques

Set the role clearly	Example: “Act as a senior test engineer.” For larger suites, role assignment should encourage categorical organization.
Provide focused context and input data	Include the Class Under Test (or relevant method), key dependencies, relevant code snippets and test scenarios that support the defined objective.
Define task instructions, constraints and output format	State exactly what the model should do, what to prioritize and which artifact is expected. Also specify framework/assertion style, required edge cases, non-goals, naming conventions and exact structure. Add examples when format consistency is required.
Choose the simplest prompting style that fits the task	Start with zero-shot for simple tasks. Use few-shot when format consistency or domain-specific behavior is needed. For reasoning-heavy tasks, consider Chain-of-Thought (CoT) and verify outputs carefully.

Guideline 02

Apply Prompt Engineering Techniques

Separate system-level and task-level instructions when helpful	Put stable behavior/persona rules in the system prompt and task-specific requests in the user prompt when persistent behavior is needed.
Batch large tasks	Split complex or multi-objective requests into smaller prompt batches instead of one overloaded prompt.
Use targeted failure context for debugging tasks	For bug localization or repair, include only the relevant traceback/error message and failing snippet instead of full logs.
Use multimodal or graph context only when it adds value	Add diagrams, UI screenshots, or graph representations only for structurally complex scenarios and keep them focused.

Guideline 02

Apply Prompt Engineering Techniques

```
System prompt: You are a senior software tester.

User prompt:
Goal: Generate PyTest unit tests for password validation

Context:
- Use the full class/module logic
- Cover scenarios:
    1) valid strong password
    2) too short
    3) missing uppercase
    4) missing number
    5) empty input

Constraints:
- Use PyTest only
- Use @pytest.mark.parametrize for repeated boundary cases
- Include both positive and negative tests
- Do not generate integration tests

Output formt:
- Return test file content only
- Test names must start with "test_"
- Add a short comment above each scenario group
```

When to Apply?

- Test generation tasks where the target behavior, framework and output format are **already defined**
- Tasks that require **consistent and executable results**
- Prompts that need a clear structure, stable formatting or a reusable template

When to Avoid?

- **Very small exploratory tasks** where a short ad-hoc prompt is faster
- Scenarios with **missing functional context, changing requirements or incomplete code**
- Cases where the prompt would become **overloaded**

Guideline 03

Evaluate, Expand, and Integrate

UNIVERSITÄT MANNHEIM

Input: LLM Generated Tests

LLM Feedback Loop

1. Use Hybrid Validation with SOTA tools
2. Feed execution errors back to the LLM for iterative script correction
3. Triangulate results across multiple LLM tools for cross-validation
4. Prompt: *What scenarios are we missing?*

Input: Target Methods + Real-World Bug Patterns

1. **Generate High-Order Mutants**
   Inject complex, real-world bug patterns into the target code to stress-test the suite.
2. **Eliminate Surviving Mutants**
   Write targeted unit tests to catch undetected bugs and close critical coverage gaps.

Core Metrics: Mutation score, Code coverage, Defect detection rate, Execution time, Test flakiness

Guideline 03

Evaluate, Expand, and Integrate

```python
def apply_discount(price, is_vip):
    if price > 100 and is_vip:
        return price * 0.8  # 20% discount
    return price

def apply_discount_mutant(price, is_vip):
    if price >= 100 or is_vip:
        return price * 0.8
    return price
```

Does this test catch all edge cases?

```python
assert apply_discount(150, True) == 120
```

When to Apply?

- LLM Feedback Loop:
  - For all AI-Generated Tests to catch hallucinations
- Mutation tests
  - When evaluating **complex conditional logic**

When to Avoid?

- When **computational or API token budgets** are limited
- When **rapid time-to-market** is prioritized

Guideline 03

Evaluate, Expand, and Integrate

```python
def apply_discount(price, is_vip):
    if price > 100 and is_vip:
        return price * 0.8  # 20% discount
    return price

def apply_discount_mutant(price, is_vip):
    if price >= 100 or is_vip:
        return price * 0.8
    return price
```

Does this test catch all edge cases?

```python
assert apply_discount(150, True) == 120
assert apply_discount(100, True) == 100
assert apply_discount(150, False) == 150
```

When to Apply?

- LLM Feedback Loop:
  - For all AI-Generated Tests to catch hallucinations
- Mutation tests
  - When evaluating **complex conditional logic**

When to Avoid?

- When **computational or API token budgets** are limited
- When **rapid time-to-market** is prioritized

Guideline 04

LLM-Generated Test Repair Loop

Smart CI/CD: AI generates and fixes tests automatically during integration.

Rapid Iteration: Runs 2–3 fix cycles based on real error logs.

Human Fallback: Only alerts developers if the AI can’t solve it.

When to Apply?

- Established test suites exist for benchmarking
- CI/CD supports iterative refinement (2-5 min budget)

When to Avoid?

- Trivial utility functions
- API costs exceed value
- Pipeline requires fast execution

Guideline 04

LLM-Generated Test Repair Loop

```yaml
test_generation_process:
  - step: generate_tests
    description: "LLM creates initial test suite based on source code."

  - step: validate_and_repair
    description: "Iterative loop (max 3 attempts) to fix failing tests."
    logic: |
      repeat up to 3 times:
        run pytest
        if success: break
        extract failures and prompt LLM for repair

  - step: human_review
    description: "Manual sign-off required if automated repair limits are reached."
```

Guideline 05

Human in the Loop

Sanitize Prompts
Manually review and remove any sensitive or confidential data.

Audit Code Integrity
Inspect the AI's output. Validate that the generated scripts strictly comply with your team's architecture rules, maintainability standards, and CI/CD pipeline requirements.

Trust the AI, but verify the output.

Now it is your turn!

Solve the tasks and explore our guidelines in practice.

Now it is your turn!

Solve the tasks and explore our guidelines in practice.

```python
def validate_password(password):
# """
# Validates a password based on:
# 1. Length between 8 and 20 characters.
# 2. Must contain at least one digit.
# 3. Must contain at least one uppercase letter.
# 4. Cannot contain the word 'password' (case-insensitive).
# """
    if not (8 <= len(password) <= 20):
        return False
    if not any(char.isdigit() for char in password):
        return False
    if not any(char.isupper() for char in password):
        return False
    if "password" in password.lower():
        return False
    return True
```

Task 1 – Define the Testing Objective

Prompt an LLM to generate a testing objective for the provided code snippet.

Task 2 – Apply Prompt Engineering

Ask an LLM to generate a prompt that you can give to your coding agent for the test generation of the provided code snippet.

Task 01 – Define the Testing Objective

Generated with Gemini 3 (Fast)

The primary objective is to **verify that the** `validate_password` **function accurately enforces the four defined security constraints** and correctly identifies both valid and invalid inputs across their respective boundaries.

Key Areas of Verification

To meet this objective, testing should be broken down into these specific goals:

- **Boundary Value Analysis (Length):** Ensure the function returns `True` for passwords of exactly **8** and **20** characters, and `False` for passwords of **7** or **21** characters.
- **Character Requirement Validation:** * Confirm it rejects strings with no digits.
  - Confirm it rejects strings with no uppercase letters.
  - Verify it accepts strings when these requirements are met at the very beginning, middle, or end of the string.
- **Keyword Restriction (Blacklist):** Verify that any variation of the word "password" (e.g., `PaSsWoRd`, `PASSWORD`, `password123`) results in a `False` return, ensuring the `.lower()` logic is working as intended.
- **Positive Confirmation:** Ensure a "clean" password that meets all criteria simultaneously (e.g., `SecureP4ss!`) returns `True`.

- Failed to specify the required **output format** or final **artifact type**
- Ignored negative paths such as **non-string data types** and **input sanitization** (whitespaces)
- Introduced "special characters" as a requirement, creating **scope creep**
- Did not define **scope boundaries** or explicitly state what is out-of-scope
- Lacks **measurable success criteria** and quality targets like branch coverage
- Omitted required **technical practices** such as parametrization and clean documentation

Task 02 – Apply Prompt Engineering

Generated with Mistral Large 3

Prompt for Test Generation:

Generate a comprehensive set of test cases for the `validate_password` function in Python. The function validates passwords based on the following rules:

1. **Length**: The password must be between 8 and 20 characters (inclusive).
2. **Digits**: The password must contain at least one digit.
3. **Uppercase**: The password must contain at least one uppercase letter.
4. **Forbidden Word**: The password must not contain the word "password" (case-insensitive).

Requirements for Test Cases:

- Include both valid and invalid passwords.
- Cover edge cases (e.g., minimum and maximum length, missing digits, missing uppercase, presence of "password").
- Provide the expected output (True or False) for each test case.
- Format the test cases as a list of tuples, where each tuple contains the password string and the expected result.

Example Format:

python Copy

```python
test_cases = [
    ("ValidPass1", True),
    ("short", False),
    ("nouppercase1", False),
    ("NoDigitsHere", False),
    ("Password123", False),
    # ... add more test cases
]
```

Additional Notes:

- Ensure the test cases are diverse and cover all validation rules.
- Include at least 10 test cases.

- Failed to set a **clear persona**
- Did **not provide** the CUT/MUT or the actual Python implementation logic, relying **only on a text description**
- Did not **separate** system-level from task-Level instructions
- Did not **specify a preferred testing framework** or **assertion style**
- No explicit **naming conventions** for the test cases or **non-goals**

Task 03 – Evaluate, Expand and Integrate

Generated with Mistral Large 3

```python
def apply_discount(price, is_vip):
    if price > 100 and is_vip:
        return price * 0.8  # 20% discount
    return price
```

Task 3

1. Prompt an LLM to generate initial test cases for the *apply_discount* function.
2. Prompt the model to identify missing scenarios.

Task 03 – Are you missing any scenarios?

Generated with Mistral Large 3

python Copy

```python
import pytest
from your_module import apply_discount  # Replace 'your_module' with the actual module name

def test_apply_discount_vip_high_price():
    """Test that a VIP gets a 20% discount for a price > 100."""
    assert apply_discount(150, True) == 120.0

def test_apply_discount_vip_low_price():
    """Test that a VIP does not get a discount for a price <= 100."""
    assert apply_discount(80, True) == 80

def test_apply_discount_non_vip_high_price():
    """Test that a non-VIP does not get a discount, even for a price > 100."""
    assert apply_discount(150, False) == 150

def test_apply_discount_non_vip_low_price():
    """Test that a non-VIP does not get a discount for a price <= 100."""
    assert apply_discount(80, False) == 80

def test_apply_discount_edge_case_price_100():
    """Test that no discount is applied when price is exactly 100, even for VIPs."""
    assert apply_discount(100, True) == 100
```

- Found the following missing scenarios
  - Negative or zero price
  - Non-numeric price
  - Non-boolean

You're right to double-check! Let's review the scenarios to ensure completeness:

Current Test Coverage

- ✅ VIP, price > 100 → Discount applied.
- ✅ VIP, price ≤ 100 → No discount.
- ✅ Non-VIP, price > 100 → No discount.
- ✅ Non-VIP, price ≤ 100 → No discount.
- ✅ Edge case: price = 100 → No discount.

Potential Missing Scenarios

1. **Negative or zero price**: What if `price` is 0 or negative? The function doesn't handle this, but it's worth considering if such inputs are possible in your use case.
2. **Non-numeric price**: What if `price` is not a number (e.g., a string)? The function assumes `price` is a number, but real-world inputs might not always be valid.
3. **Non-boolean `is_vip`**: What if `is_vip` is not a boolean (e.g., `None`, a string, or an integer)? The function assumes it's a boolean.

Thanks for your attention!

Any questions?

References

For complete references refer to the guideline package.

- SAP. (2024, August 8). *What is prompt engineering?* https://www.sap.com/resources/what-is-prompt-engineering
- Santana, D., Magalhaes, C., & de Souza Santos, R. (2025, November). Software Testing with Large Language Models: An Interview Study with Practitioners. In 2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware) (pp. 96-104). IEEE.
- Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering, 50(4), 911-936.
- Zhang, Q., Fang, C., Gu, S., Shang, Y., Chen, Z., & Xiao, L. (2025). Large language models for unit testing: A systematic literature review. arXiv preprint arXiv:2506.15227.